

Technologie Obiektowe

Projekt

Temat: Grywalizacja - Aplikacja do nauki dla studenta

Wykonał: Paweł Kusztal
Grupa: 1ID21A

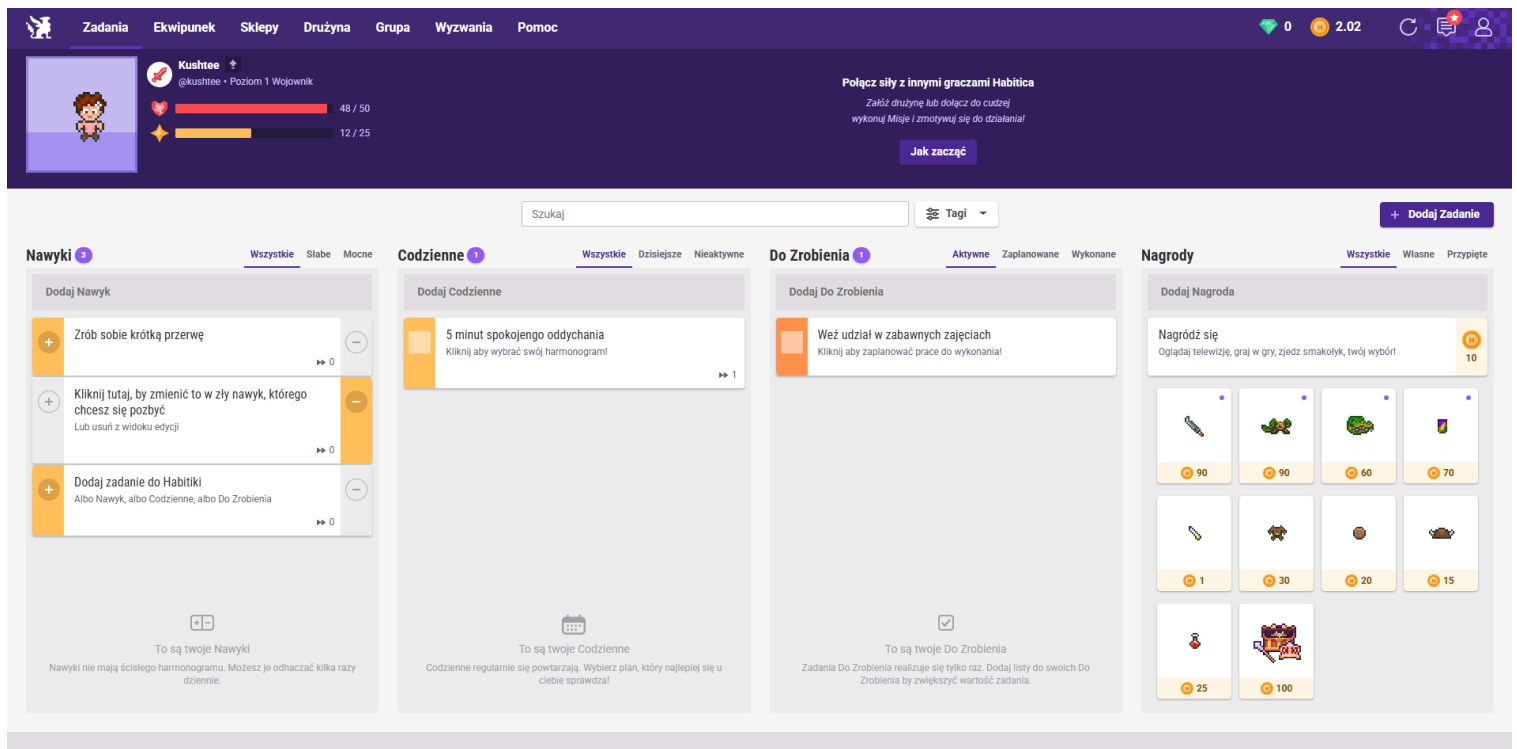
1. Wstęp

Tematyką projektu było stworzenie aplikacji do nauki z elementami grywalizacji. Aplikacja powinna pozwalać na dodawanie kursów, tworzenie zadań oraz ich zaliczanie przez użytkownika. Dodatkowo powinna śledzić postępy użytkownika w danym kursie, pozwalać na rozwiązywanie testów, a także posiadać elementy grywalizacji takie jak poziomy, punkty doświadczenia, bonusy, osiągnięcia.

Grywalizacja (z ang. gamification, zwana też gamifikacją lub gryfikacją) to zastosowanie mechanizmów i technik projektowania gier w kontekstach niezwiązanych z grami — w celu angażowania ludzi, zmiany ich zachowań i osiągania konkretnych celów. [1] Grywalizacja może być wykorzystywana w aplikacjach do nauki w sposób, który pozwoli na łatwiejsze przyswajanie wiedzy, a także ułatwi wykonywanie rutynowych zadań.

Stworzona aplikacja czerpie inspirację z podobnych rozwiązań dostępnych dla użytkowników w internecie. Oto przykładowe aplikacje wykorzystujące technikę grywalizacji:

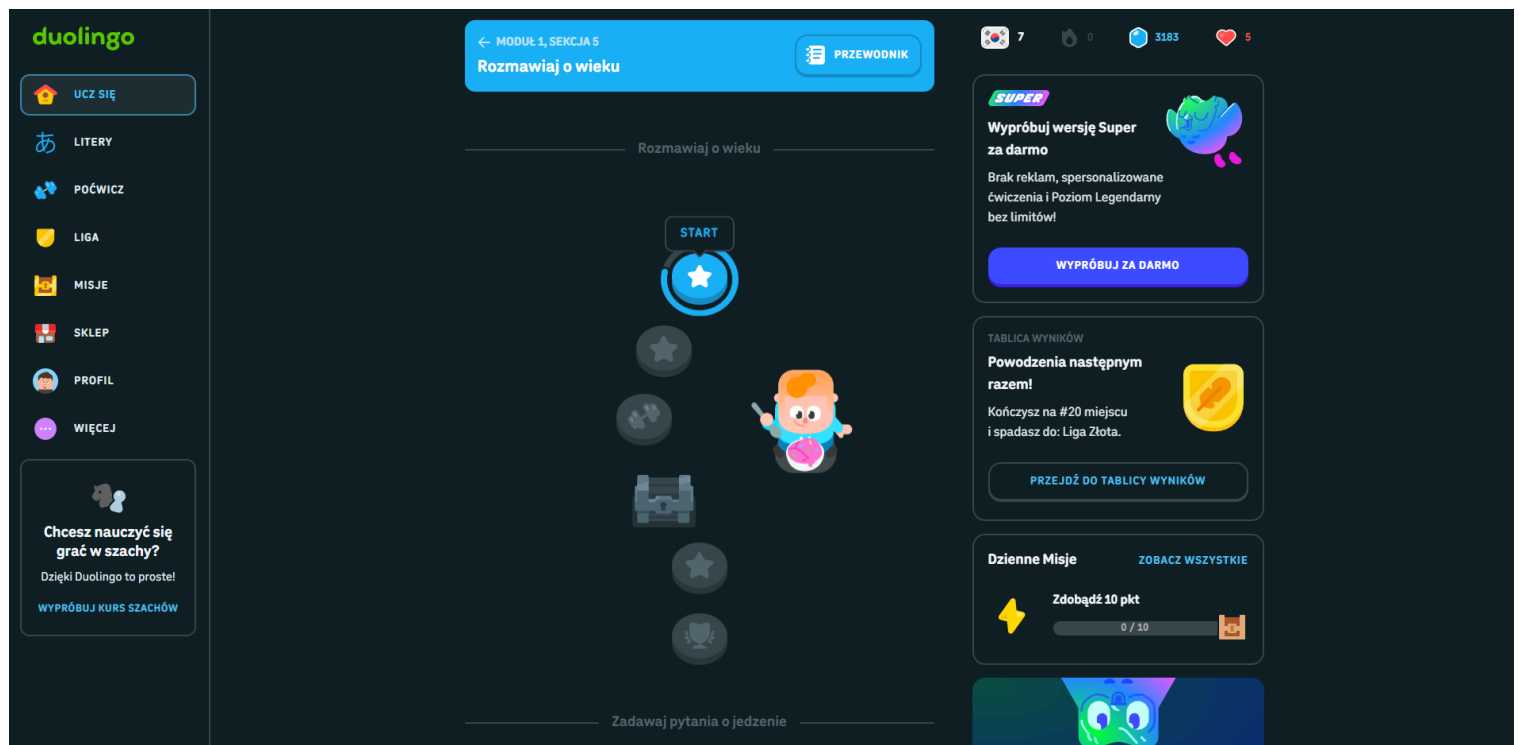
- Habitica - habitica.com



Strona główna aplikacji Habitica

Aplikacja ta pozwala na tworzenie zadań, za realizację których otrzymuje punkty doświadczenia, złoto oraz inne nagrody, które pozwalają na rozwój postaci. Wykorzystane przez aplikację mechanizmy to między innymi system poziomów, walki z bossami, elementy społecznościowe.

- Duolingo - duolingo.com



Strona główna aplikacji Duolingo

Duolingo to platforma edukacyjna do nauki języków. Wykorzystuje ona takie mechanizmy grywalizacji jak poziomy, punkty doświadczenia, serie dni nauki, rankingi oraz ligi. Nauka języka podzielona jest w niej na krótkie lekcje. Aplikacja zawiera również wersję premium, która zapewnia dodatkowe bonusy oraz wyższe nagrody za ukończenie lekcji.

2. Wykorzystane technologie i narzędzia

Do stworzenia aplikacji wykorzystałem następujące technologie:

- JavaScript - język programowania użyty do stworzenia aplikacji klienta
- React - biblioteka JavaScript służąca do budowy interfejsów użytkownika w oparciu o komponenty
- HTML - język znaczników służący do tworzenia struktury stron internetowych
- CSS - język arkuszy stylów używany do opisywania wyglądu stron internetowych
- react-icons - biblioteka udostępniająca zestaw gotowych ikon do wykorzystania w aplikacjach React.
- react-toastify - biblioteka do wyświetlania powiadomień typu „toast” w aplikacjach React
- Java - obiektowy język programowania, wykorzystany do stworzenia części backendowej projektu
- Spring - framework dla języka Java, ułatwiający tworzenie aplikacji backendowych, zapewnia obsługę REST API, zarządzanie zależnościami oraz integrację z bazami danych

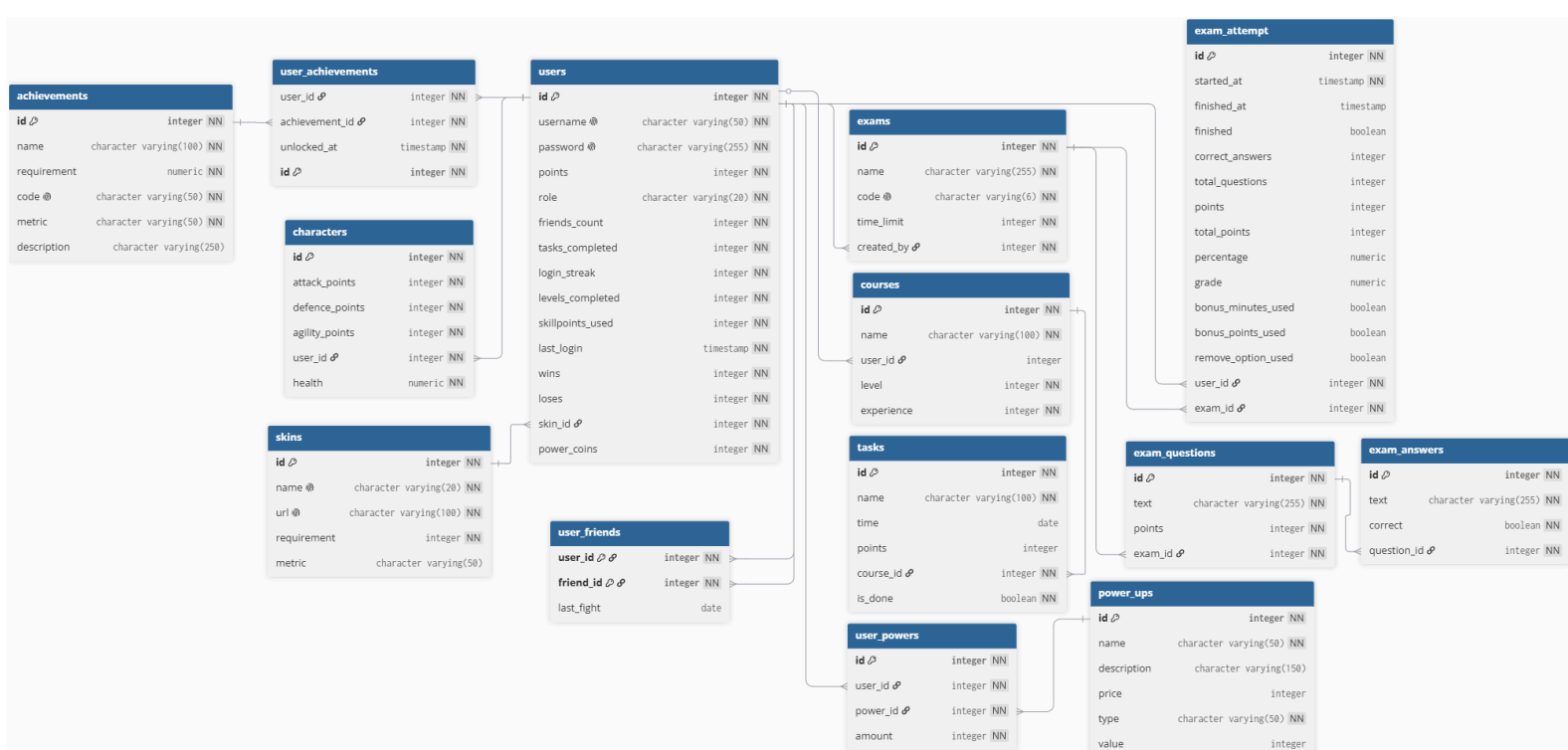
- PostgreSQL - relacyjny system zarządzania bazą danych, wykorzystywany do przechowywania danych aplikacji
- Visual Studio Code - edytor kodu źródłowego firmy Microsoft
- IntelliJ IDEA - zintegrowane środowisko programistyczne (IDE) stworzone przez firmę JetBrains

3. Założenia aplikacji

Projekt zakłada realizację następujących funkcjonalności:

- zarządzanie użytkownikami - aplikacja umożliwia rejestrację użytkowników oraz logowanie do systemu
- system kursów i zadań - użytkownicy mają możliwość tworzenia lub dołączania do istniejących kursów oraz dodawania i wykonywania zadań
- system grywalizacji - mechanizmy zdobywania punktów doświadczenia, awansowanie na kolejne poziomy w kursach, zdobywanie osiągnięć, zdobywanie monet za stoczone pojedynki, wydawanie monet na bonusy ułatwiające rozwiązywanie egzaminów
- system postaci użytkownika - do użytkownika przypisana jest postać, którą można rozwijać poprzez ulepszanie jej statystyk, użytkownik ma również możliwość dostosowywania wyglądu postaci
- dodawanie znajomych i wyzywanie na pojedynki - aplikacja umożliwia dodawanie innych użytkowników do listy znajomych oraz wyzywanie ich na pojedynki
- system egzaminów - użytkownicy mogą rozwiązywać egzaminy sprawdzające wiedzę po podaniu odpowiedniego kodu egzaminu, w trakcie rozwiązywania mają możliwość wykorzystania bonusu, aby ułatwić sobie test
- role użytkowników - trzy role użytkowników o różnym zakresie dostępu do aplikacji: zwykły użytkownik, nauczyciel oraz admin

4. Schemat bazy danych



5. Implementacja

System składa się z trzech warstw: frontend - aplikacja klienta, backend - aplikacja serwera oraz baza danych. Komunikacja między klientem, a serwerem odbywa się poprzez REST API (HTTP/JSON).

5.1 Serwer

Warstwa serwerowa została zaimplementowana w technologii Spring Boot i oparta jest na wielowarstwowej architekturze, pozwalającej na rozdzielenie odpowiedzialności poszczególnych komponentów systemu. Warstwę odpowiedzialną za obsługę żądań HTTP od klienta i wysyłanie odpowiedzi stanowią kontrolery. Udostępniają one endpointy, poprzez które klient może wykonywać operacje. Przykładowe kontrolery: CharacterController, ExamController, TaskController, UserController.

```
@RestController  Paweł Kusztal *
@RequestMapping("/achievements")
public class AchievementController {
    private final AchievementService achievementService; 4 usages
    private final UserRepository userRepository; 2 usages

    public AchievementController(AchievementService achievementService, UserRepository userRepository) { no usages
        this.achievementService = achievementService;
        this.userRepository = userRepository;
    }

    @PostMapping  Paweł Kusztal
    public ResponseEntity<AchievementDTO> createAchievement(@RequestBody AchievementRequestDTO dto) {
        Achievement achievement = achievementService.createAchievement(AchievementMapper.toEntity(dto));

        return ResponseEntity.status(HttpStatus.CREATED).body(AchievementMapper.toDTO(achievement));
    }

    @GetMapping  Paweł Kusztal *
    public ResponseEntity<List<AchievementDTO>> getAllUserAchievements(Authentication authentication) {
        return ResponseEntity.ok(AchievementMapper.toDTOList(achievementService
            |.getUserAchievements(userRepository.findByUsername(authentication.getName()).orElseThrow())));
    }

    @GetMapping("/all")  Paweł Kusztal
    public ResponseEntity<List<AchievementDTO>> getAllAchievements(Authentication authentication) {
        return ResponseEntity.ok(AchievementMapper.achievementsToDTOList(achievementService.getAllAchievements()));
    }
}
```

Klasa AchievementController z warstwy kontrolerów

Logika biznesowa wydzielona została do warstwy serwisów. To w niej realizowane są mechanizmy takie jak przyznawanie punktów doświadczenia, przetwarzanie wyników egzaminu, weryfikowanie przyjętych przez kontrolery danych czy system pojedynków między użytkownikami. Dzięki temu logika została całkowicie odseparowana od kontrolerów. Przykłady serwisów w projekcie: AchievementsService, CharacterService, CourseService, PowerUpService, TaskService, UserService.

```
@Transactional 1 usage  Paweł Kusztal
public Character addStatPoint(String username, StatType stat) {
    User user = userRepository.findByUsername(username).orElseThrow();

    if (user.getPoints() <= 0) {
        throw new RuntimeException(HttpStatus.BAD_REQUEST, "Not enough points");
    }

    Character character = user.getCharacter();

    switch (stat) {
        case ATTACK -> character.setAttackPoints(character.getAttackPoints() + 1);
        case DEFENCE -> character.setDefencePoints(character.getDefencePoints() + 1);
        case AGILITY -> character.setAgilityPoints(character.getAgilityPoints() + 1);
        case HEALTH -> character.setHealth(character.getHealth() + 5);
    }

    user.setPoints(user.getPoints() - 1);

    characterRepository.save(character);

    userService.incrementStat(user, AchievementMetric.SKILLPOINTS_USED);
    publisher.publishEvent(new ProgressEvent(user.getId(), AchievementMetric.SKILLPOINTS_USED, user.getSkillpointsUsed()));

    return character;
}
```

Metoda addStatPoint w klasie CharacterService obsługująca dodawanie punktów statystyk bohatera

Dostęp do bazy danych realizowany jest za pomocą repozytoriów opartych na Spring Data JPA. Odpowiadają one za komunikację z bazą danych oraz wykonywanie zapytań bez konieczności ręcznego tworzenia ich w języku SQL. Przykładowe repozytoria: AchievementRepository, CharacterRepository, ExamRepository, SkinRepository, UserRepository.

```
@Repository 8 usages  Paweł Kusztal
public interface TaskRepository extends JpaRepository<Task, Long>{
    List<Task> findById(Long courseId); 1 usage  Paweł Kusztal
    List<Task> findByCourse(Course course); 1 usage  Paweł Kusztal
}
```

Klasa TaskRepository

Model danych został odwzorowany za pomocą encji (Entities) odpowiadających bazie danych. Przykładowe encje w warstwie modelu: Achievement, Character, Course, Task, User.

```
@Entity  👤 Paweł Kuształ *
@Table(name = "exams")
public class Exam {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false) 2 usages
    private String name;

    @Column(length = 6, nullable = false, unique = true) 2 usages
    private String code;

    @Column(nullable = false) 2 usages
    private Integer timeLimit;

    @ManyToOne 2 usages
    @JoinColumn(name = "created_by")
    private User createdBy;

    public Long getId() { return id; }

    public String getName() { return name; }

    public void setName(String name) { this.name = name; }

    public String getCode() { return code; }

    public void setCode(String code) { this.code = code; }

    public Integer getTimeLimit() { return timeLimit; }

    public void setTimeLimit(Integer timeLimit) { this.timeLimit = timeLimit; }

    public User getCreatedBy() { return createdBy; }

    public void setCreatedBy(User createdBy) { this.createdBy = createdBy; }
}
```

Klasa Exam modelująca encję exams z bazy danych

Dodatkowo, aby nie odsłaniać klientowi bezpośrednio modelu zastosowano obiekty DTO - Data Transfer Object. Pozwalają one na kontrolę zakresu danych przekazywanych pomiędzy serwerem, a klientem. Pozwala to na ograniczenie wielkości odpowiedzi wysłanej do klienta oraz zmniejsza ryzyko niepotrzebnego ujawnienia wewnętrznej struktury danych. Przykładowe obiekty: AuthResponseDTO, CharacterDTO, ExamQuestionDTO, ExamResultDTO, UserProgressDTO, UserResponseDTO.

```
public record CharacterDTO( 8 usg
    Integer attackPoints, no
    Integer defencePoints,
    Integer agilityPoints,
    Float health) { no usages
}
```

CharacterDTO z informacjami o statystykach bohatera

Aby uprościć tworzenie obiektów DTO z obiektów encji oraz odwrotnie zastosowano pomocnicze statyczne klasy mapper, które mapują obiekty w odpowiedni sposób. Przykładowe mapery: AchievementMapper, CourseMapper, ExamMapper, UserMapper.

```
public class TaskMapper { 9 usages  Paweł Kusztal *
    private TaskMapper() {} no usages  Paweł Kusztal

    public static Task toEntity(TaskRequestDTO dto) {
        Task task = new Task();
        task.setName(dto.name());
        task.setPoints(dto.points());
        task.setDone(dto.isDone());
        return task;
    }

    public static TaskResponseDTO toDTO(Task task) {
        return new TaskResponseDTO(
            task.getId(),
            task.getName(),
            task.getTime(),
            task.getPoints(),
            task.getCourse().getId(),
            task.getDone());
    }
}
```

Klasa TaskMapper z metodami toEntity oraz toDTO

5.2 Klient

Warstwa klienta została zaimplementowana z wykorzystaniem biblioteki React. Aplikacja została zbudowana z komponentów, dzięki czemu interfejs użytkownika podzielony jest na mniejsze elementy odpowiedzialne za realizację konkretnych funkcji. Podział na komponenty pozwala również na łatwiejsze wielokrotne używanie elementów powtarzających się w interfejsie. Komunikacja z warstwą serwera została wydzielona do odpowiednich metod API. W interfejsie przechodzenie pomiędzy poszczególnymi elementami tworzonymi przez komponenty zostało stworzone przy użyciu stanów, co pozwala na wyświetlanie różnych ekranów interfejsu pod tym samym adresem. Każdy komponent posiada również funkcje odpowiedzialne za obsługę zdarzeń w tych komponentach. W projekcie wykorzystano również dwa własne hooki React, useCourses oraz useTasks. Dzięki temu obsługa zadań oraz zarządzanie danymi w Zadaniach i Kursach jest odpowiednio wydzielona. W aplikacji zastosowano również React Routing do stworzenia podstron logowania, rejestracji, administratora. Za wygląd aplikacji odpowiadają arkusze stylów CSS.

```
export default function Home({ user, setUser }) {  
  return (  
    <div className="page">  
      <Header user={user} setUser={setUser} points={points} setActiveView={setActiveView} selectedCourse={selectedCourse} setSelectedCourse={setSelectedCourse} />  
      <main>  
        {user ? (  
          <>  
            <div className="main-content">  
              {activeView === "home" && (  
                <HomeContent user={user} selectedCourse={selectedCourse} setSelectedCourse={setSelectedCourse} points={points} setPoints={setPoints} c />  
              )}  
              {activeView === "achievements" && (  
                <Achievements user={user} setUser={setUser} setActiveView={setActiveView} />  
              )}  
              {activeView === "arena" && (  
                <Arena user={user} setUser={setUser} character={character} friend={friendToFight} setActiveView={setActiveView} setCoins={setCoins} />  
              )}  
              {activeView === "leaderboard" && (  
                <Leaderboard user={user} character={character} friend={friendToFight} setActiveView={setActiveView} />  
              )}  
              {activeView === "customization" && (  
                <Customization user={user} setUser={setUser} setActiveView={setActiveView} />  
              )}  
              {activeView === "exams" && (  
                <ExamPage user={user} setUser={setUser} setCurrentExamAttempt={setCurrentExamAttempt} setActiveView={setActiveView} powerUps={powerUps} />  
              )}  
              {activeView === "exam" && (  
                <ExamSession user={user} currentExamAttempt={currentExamAttempt} setActiveView={setActiveView} selectedAnswers={selectedAnswers} setSe />  
              )}  
            </div>  
            <Friends user={user} setActiveView={setActiveView} setFriendToFight={setFriendToFight} />  
          </>  
        ) : (  
          <h2 className="no-login-text">  
            Aby korzystać z aplikacji należy się zalogować  
          </h2>  
        )  
      </main>  
    </div>  
  );  
}
```

Home.js odpowiadający za zawartość strony głównej

6. Instrukcja uruchomienia i prezentacja aplikacji

Aby uruchomić aplikację należy w folderze client-app w konsoli wpisać komendę npm run start. W przeglądarce powinna otworzyć się nowa karta z aplikacją. Jeżeli wyskoczy błąd związany z brakiem bibliotek należy je zainstalować.

```
C:\Users\pawel\Desktop\Psk\T0\Projekt\client-app>npm run start|
```

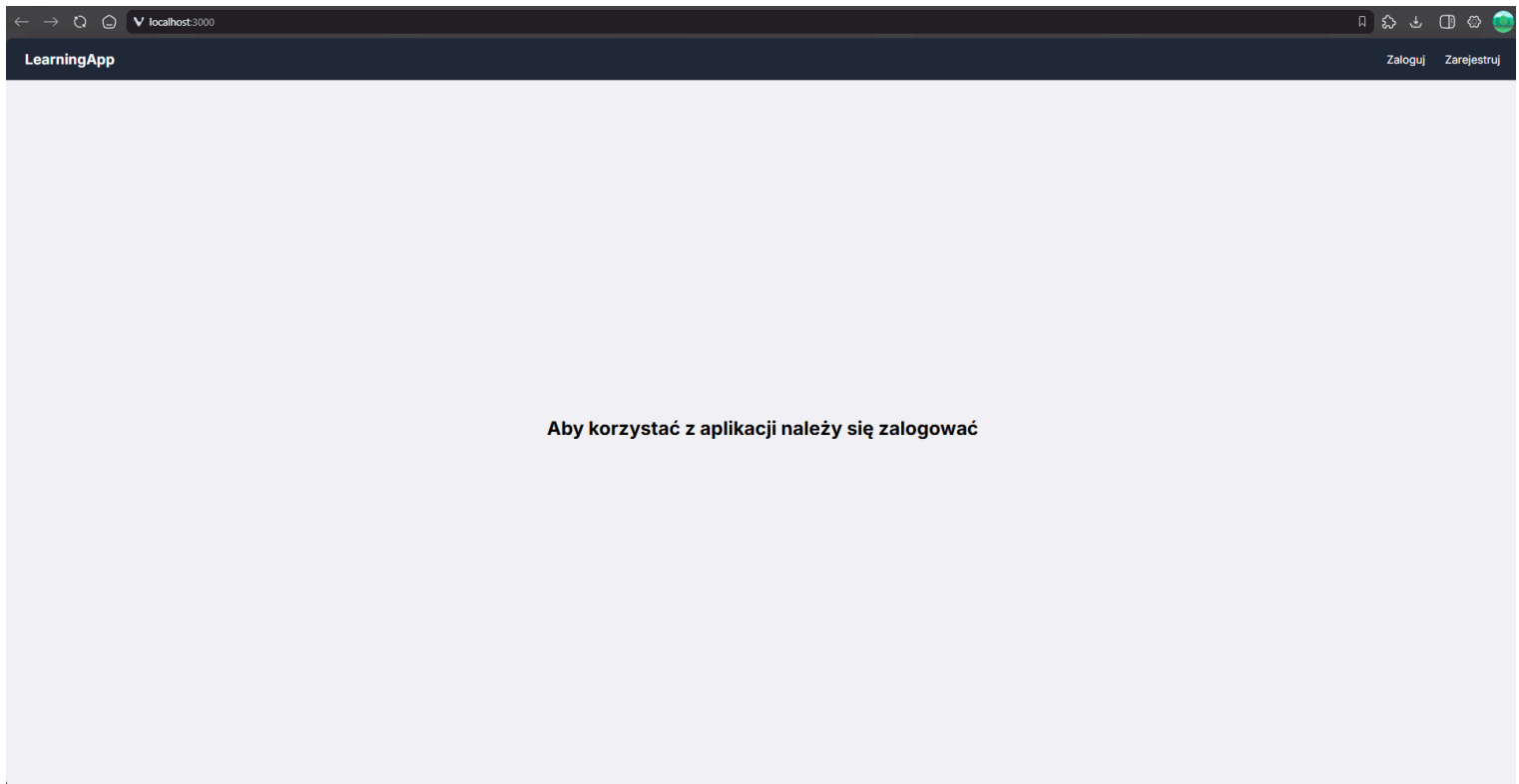
Komenda uruchamiająca klienta

Aby uruchomić serwer należy w folderze Server wykonać w konsoli polecenie mvn spring-boot:run. Innym sposobem na uruchomienie aplikacji jest otwarcie projektu Server w IDE np. IntelliJ i uruchomienie w nim pliku ServerApplication.java.

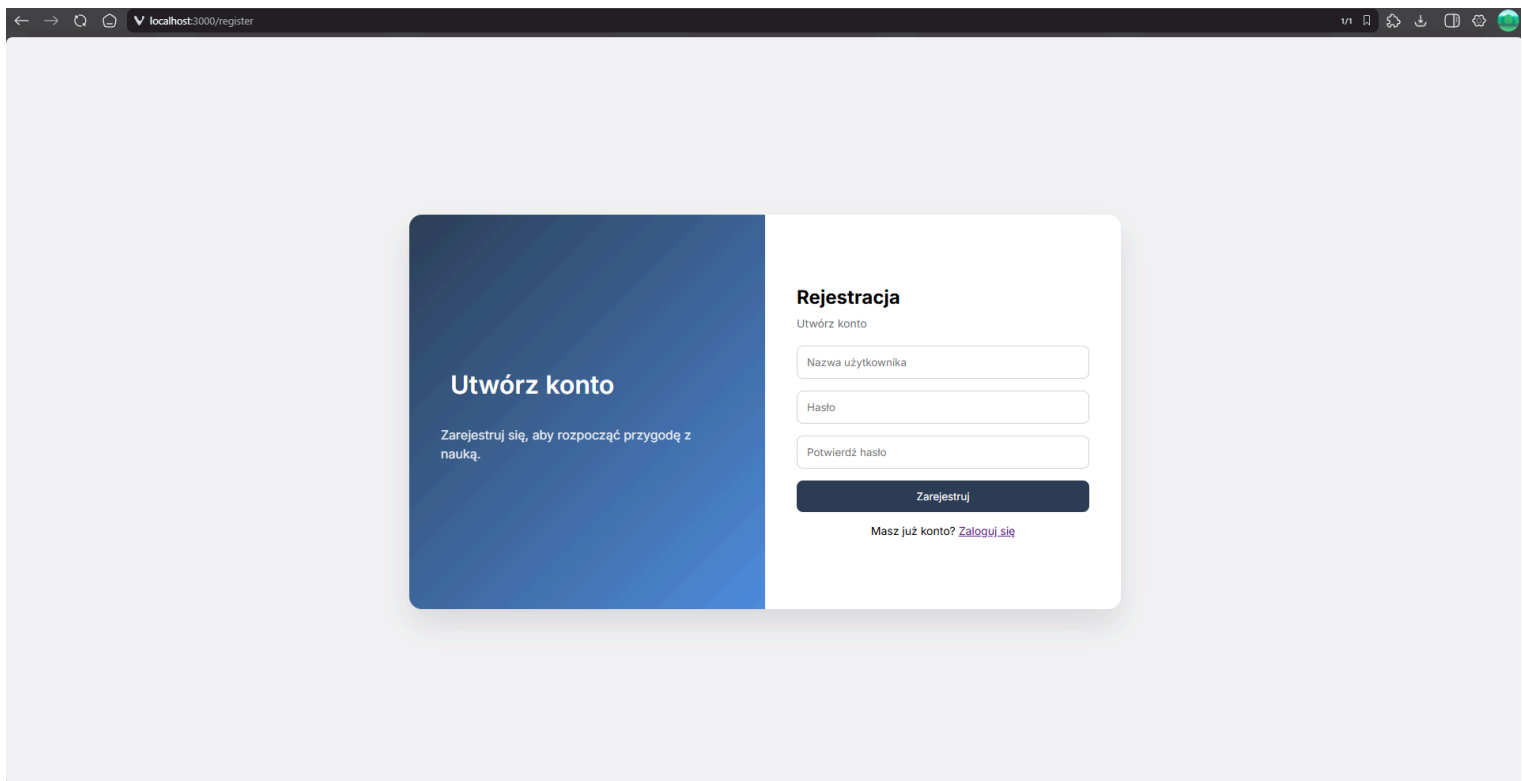
```
C:\Users\pawel\Desktop\Psk\T0\Projekt\Server>mvn spring-boot:run|
```

Komenda uruchamiająca serwer

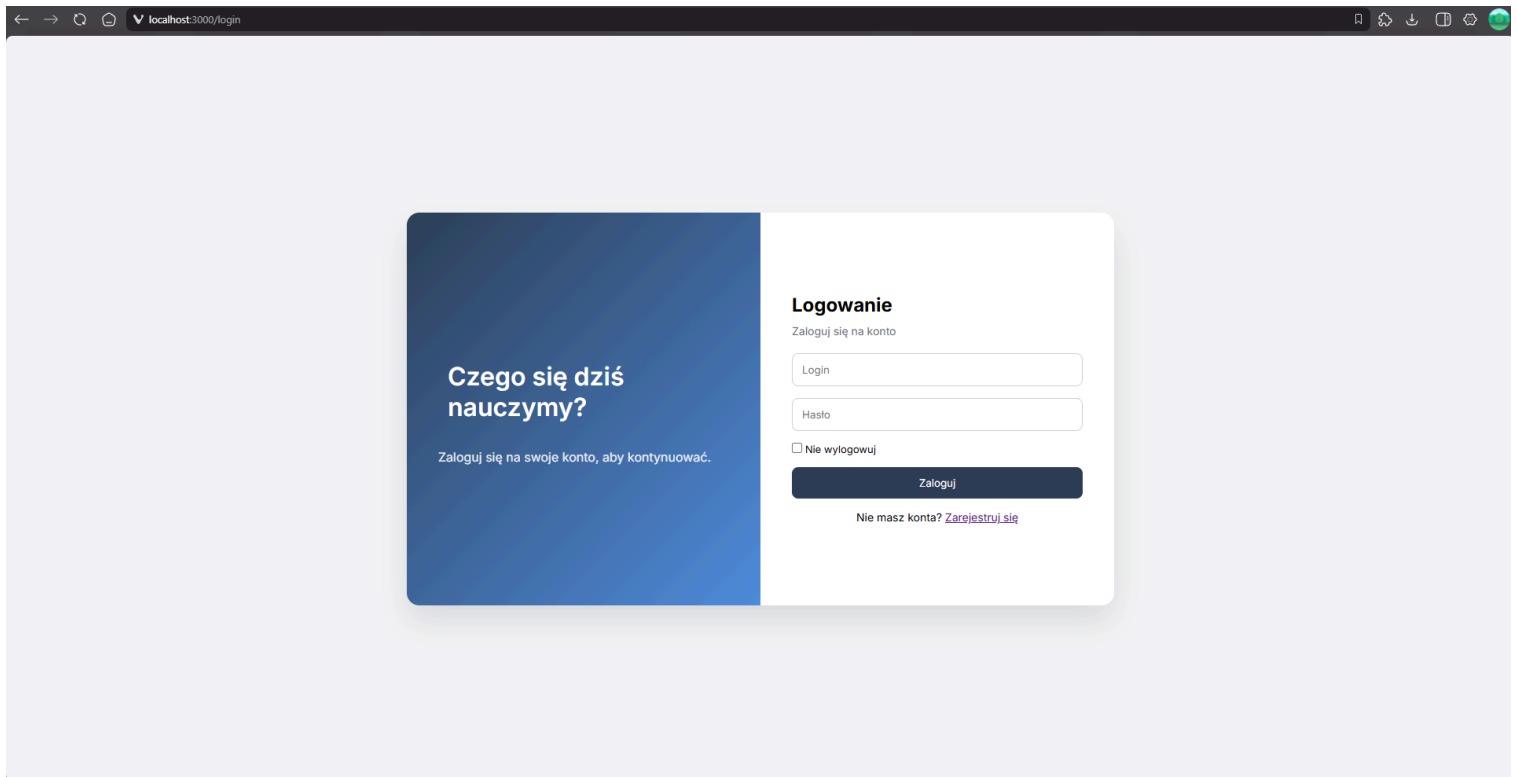
Aby aplikacja działała wymagane jest PostgreSQL, ze stworzoną w niej bazą danych o nazwie learning_db. Następnie należy dodać do bazy do tabeli skins przynajmniej jeden rekord. Po tych krokach można uruchomić aplikację, zarejestrować użytkownika, a następnie się zalogować. Aby móc używać konta Admin lub Teacher, po zarejestrowaniu w bazie danych należy zmienić w tabeli users w kolumnie role wartość odpowiednio na TEACHER lub ADMIN.



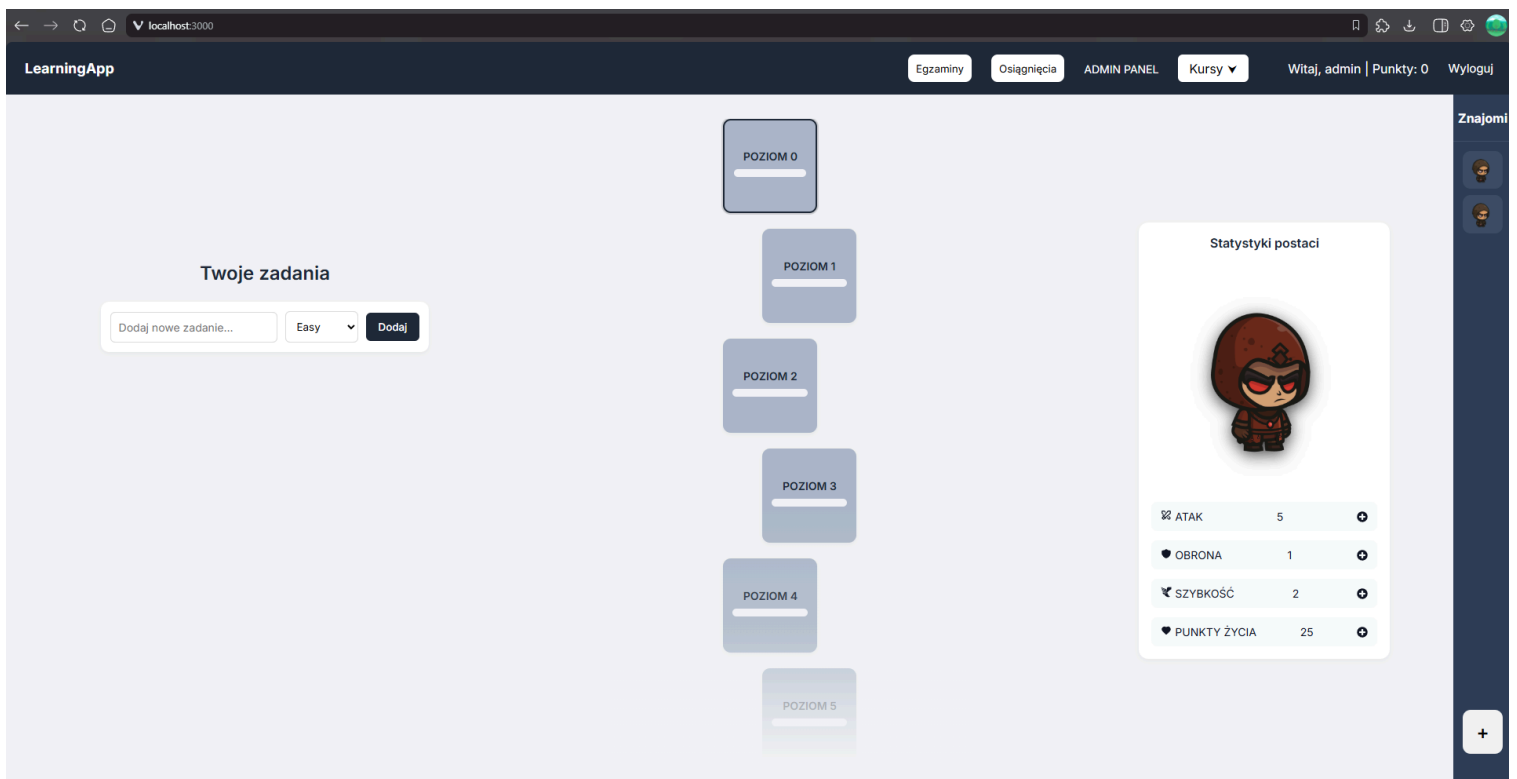
Strona główna aplikacji bez zalogowania



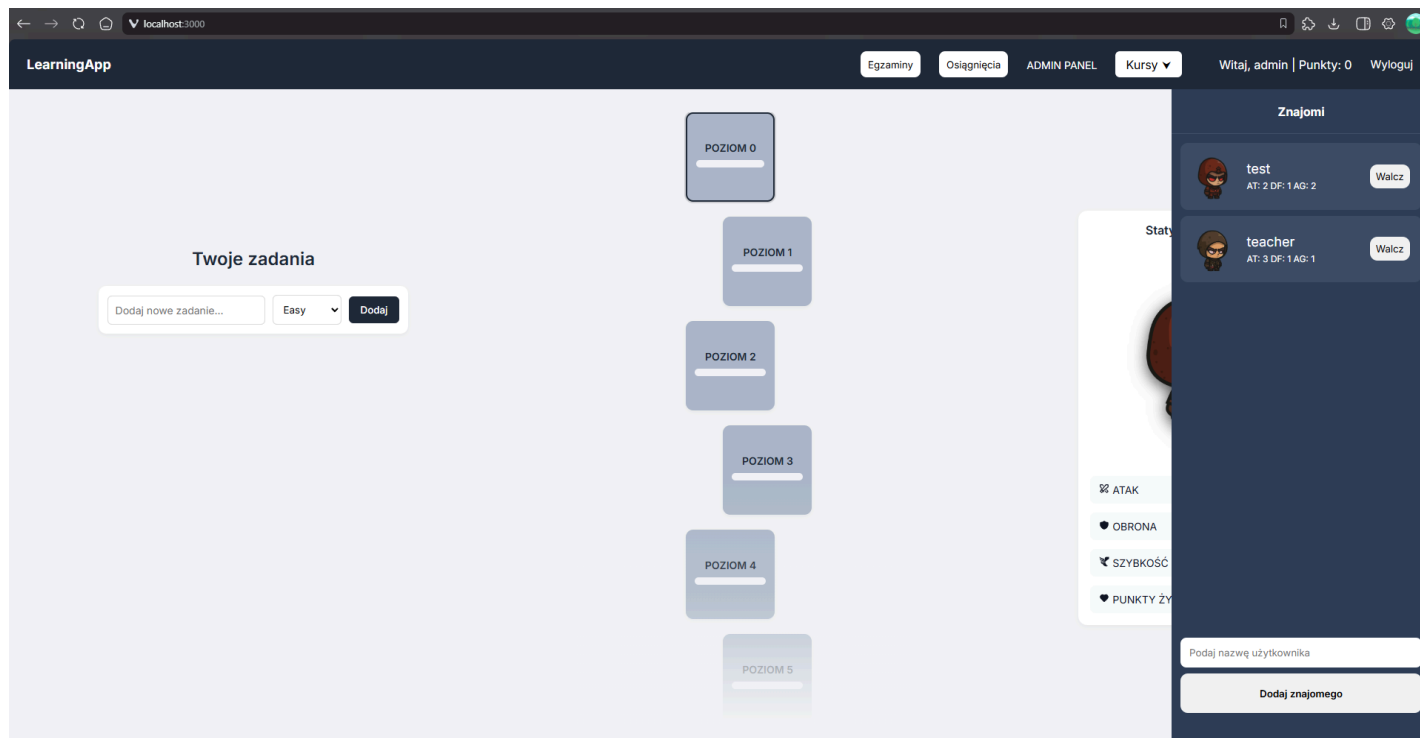
Strona rejestracji użytkownika



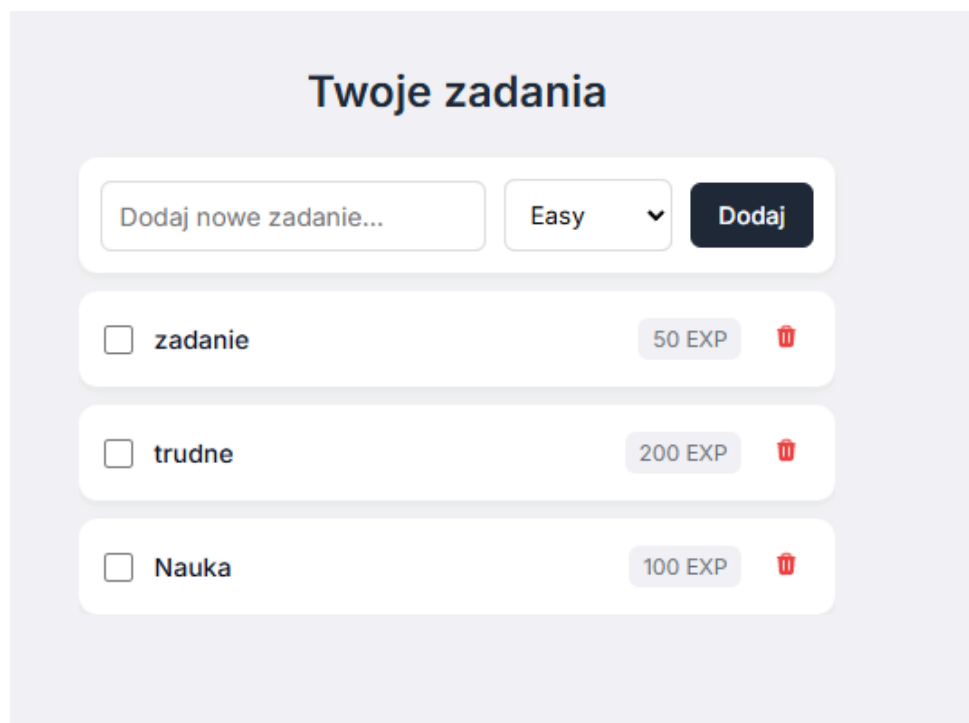
Strona logowania



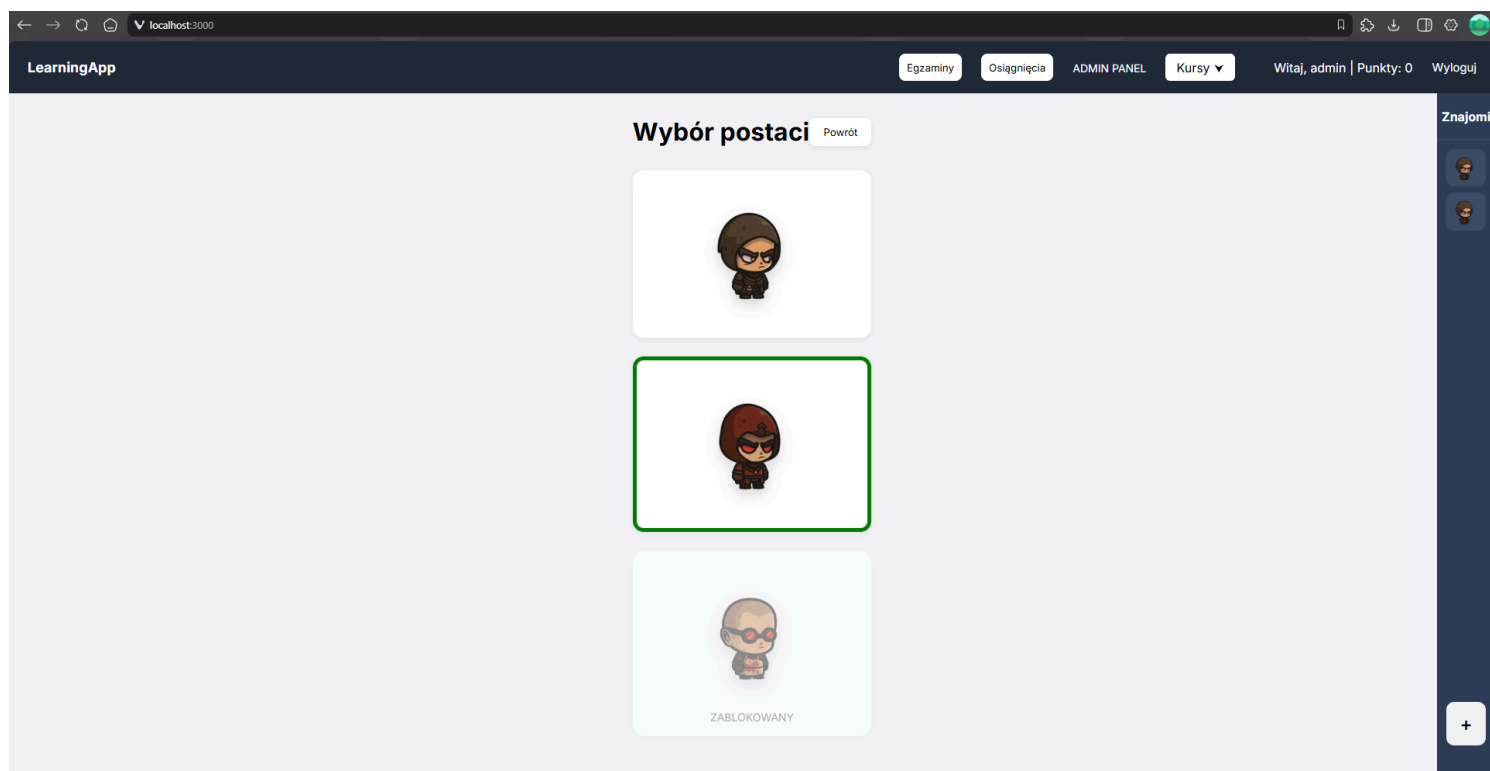
Strona główna po zalogowaniu



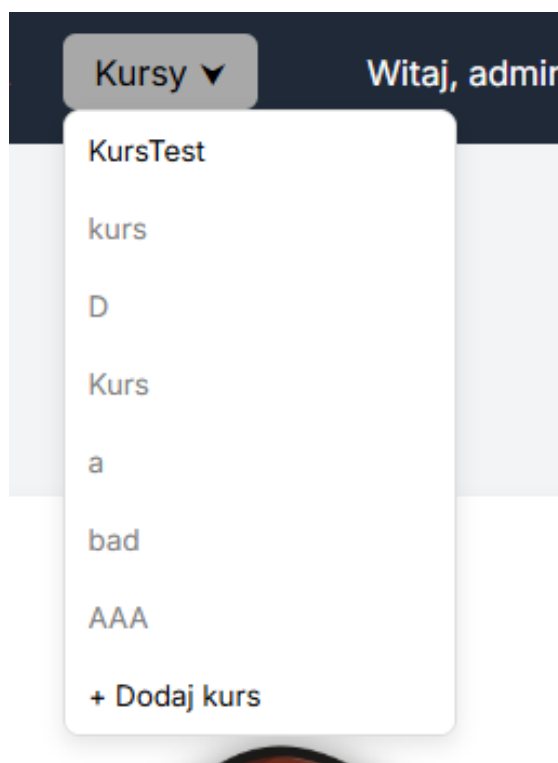
Strona główna z rozwiniętą po prawej stronie listą znajomych



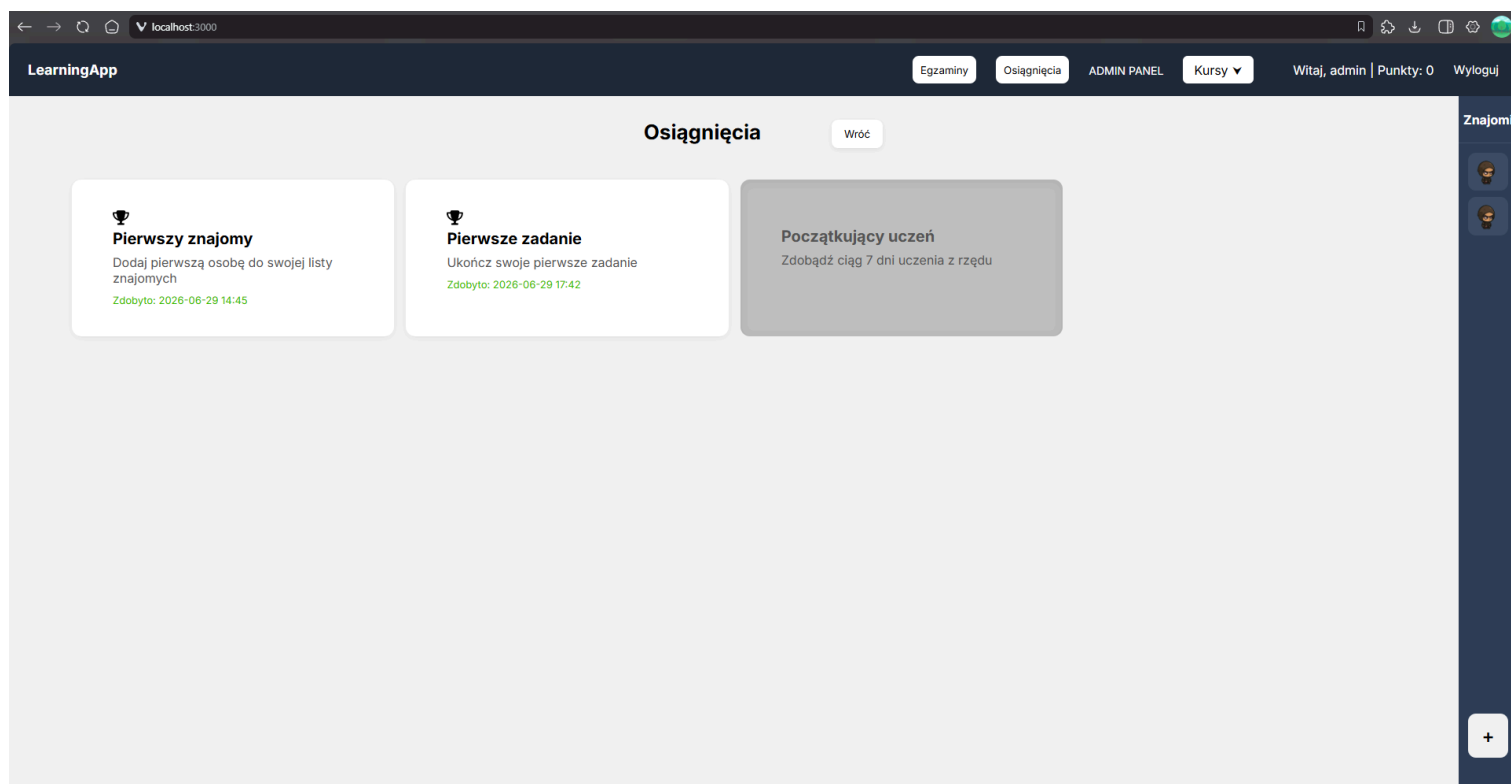
Lista zadań



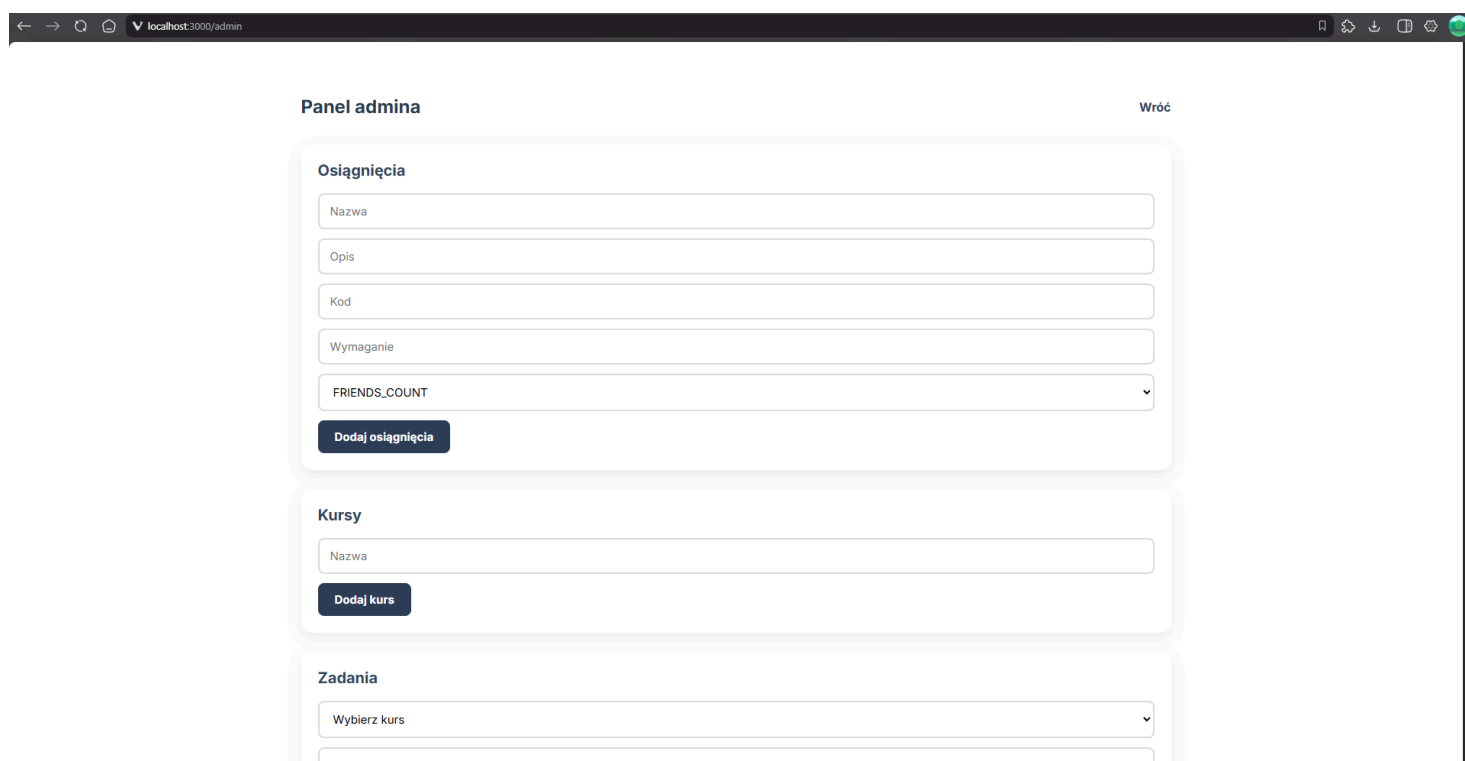
Strona wyboru wyglądu postaci



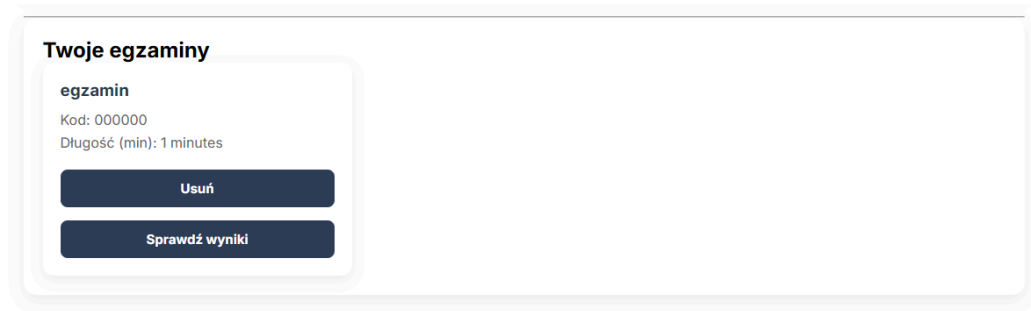
Rozwinięta lista dostępnych kursów



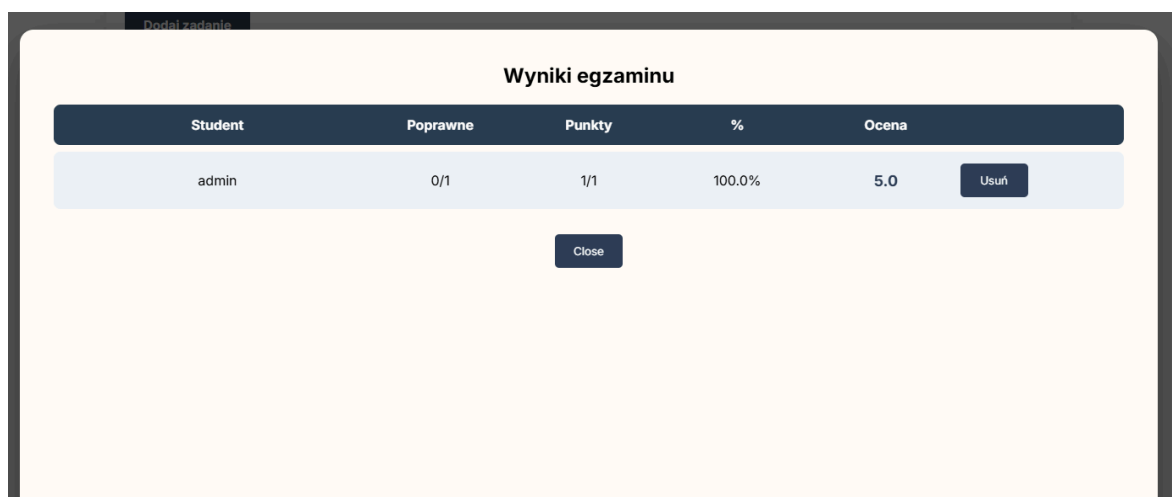
Osiągnięcia użytkownika



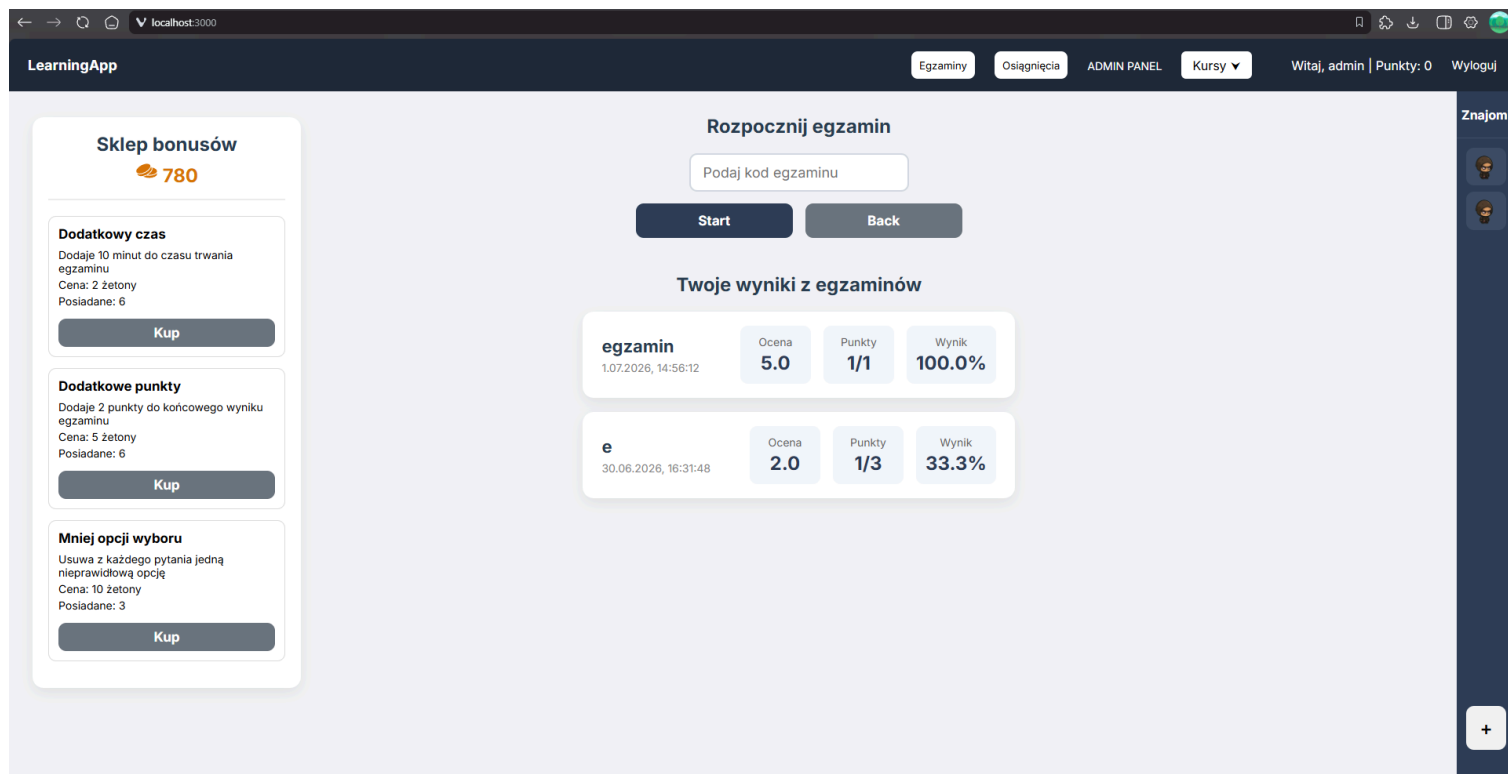
Strona panelu administratora



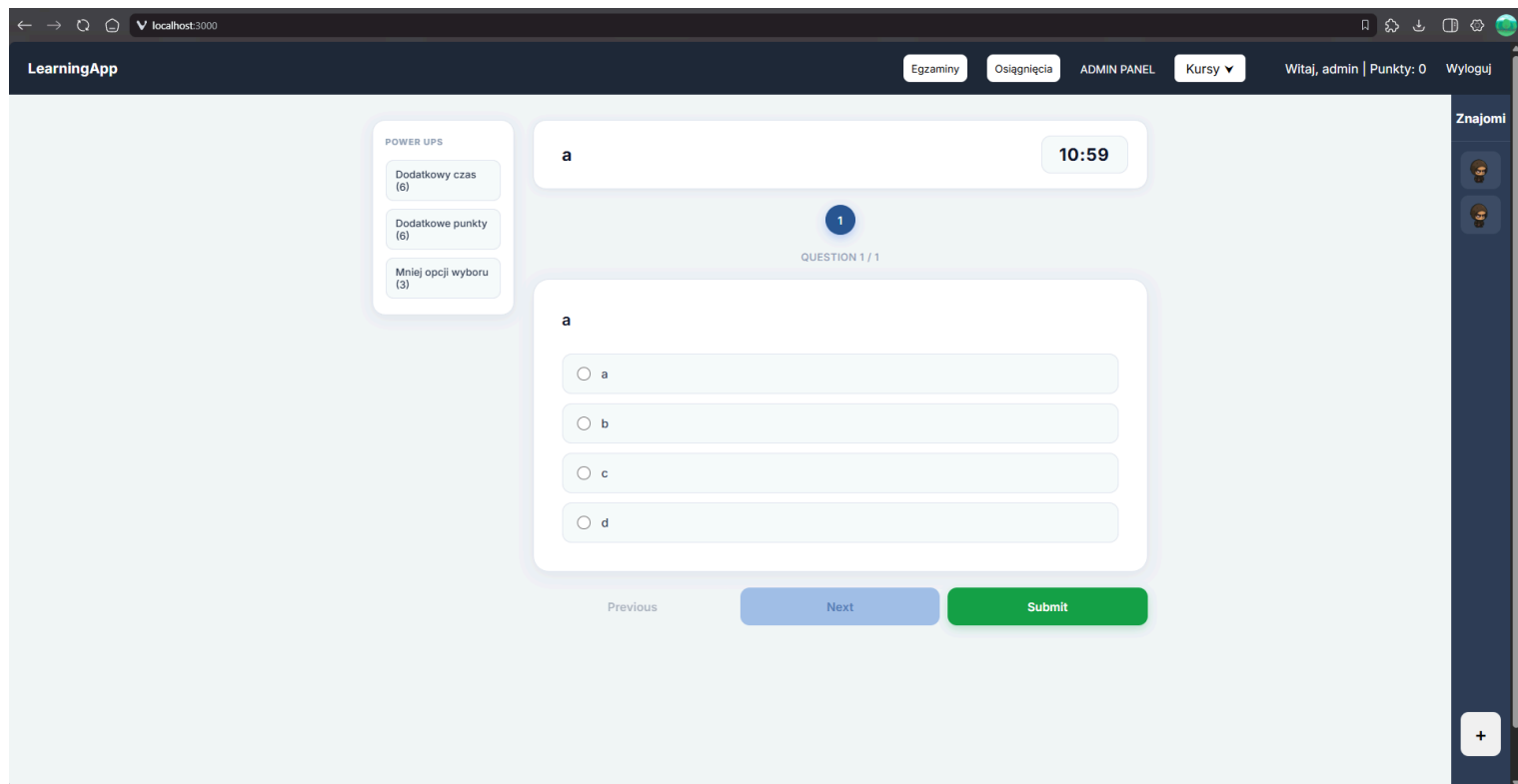
Element z egzaminami, które utworzył użytkownik na koncie admina/nauczyciela



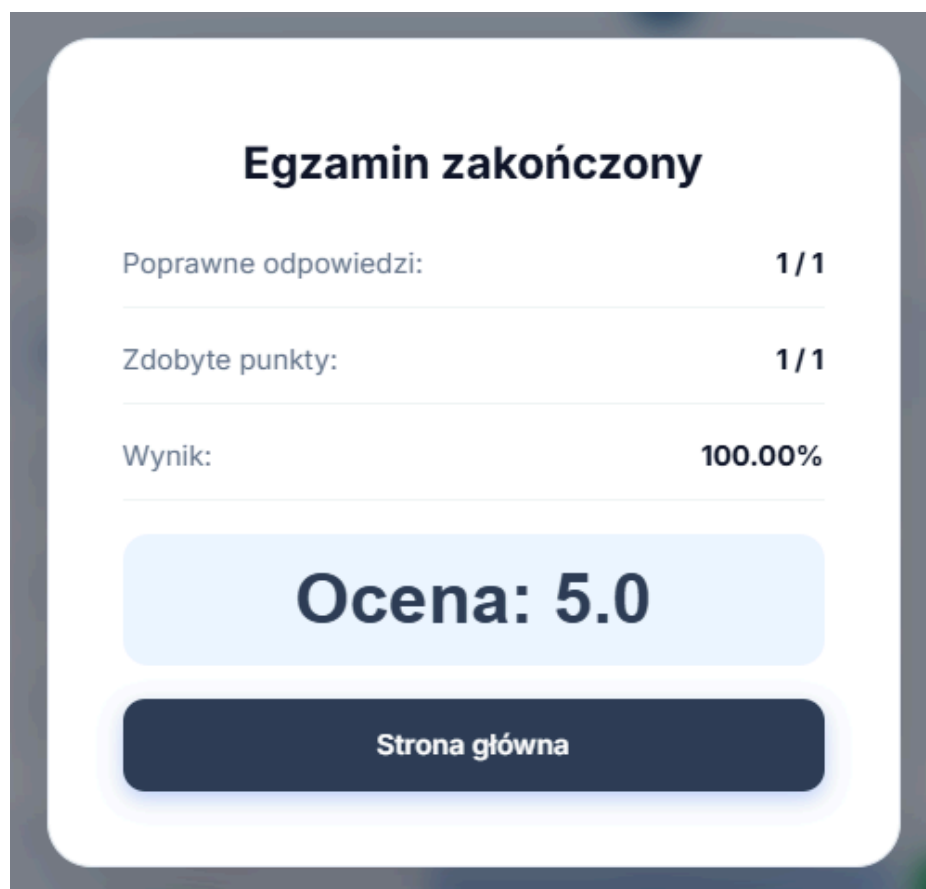
Okno z wynikami egzaminu utworzonego przez admina/nauczyciela



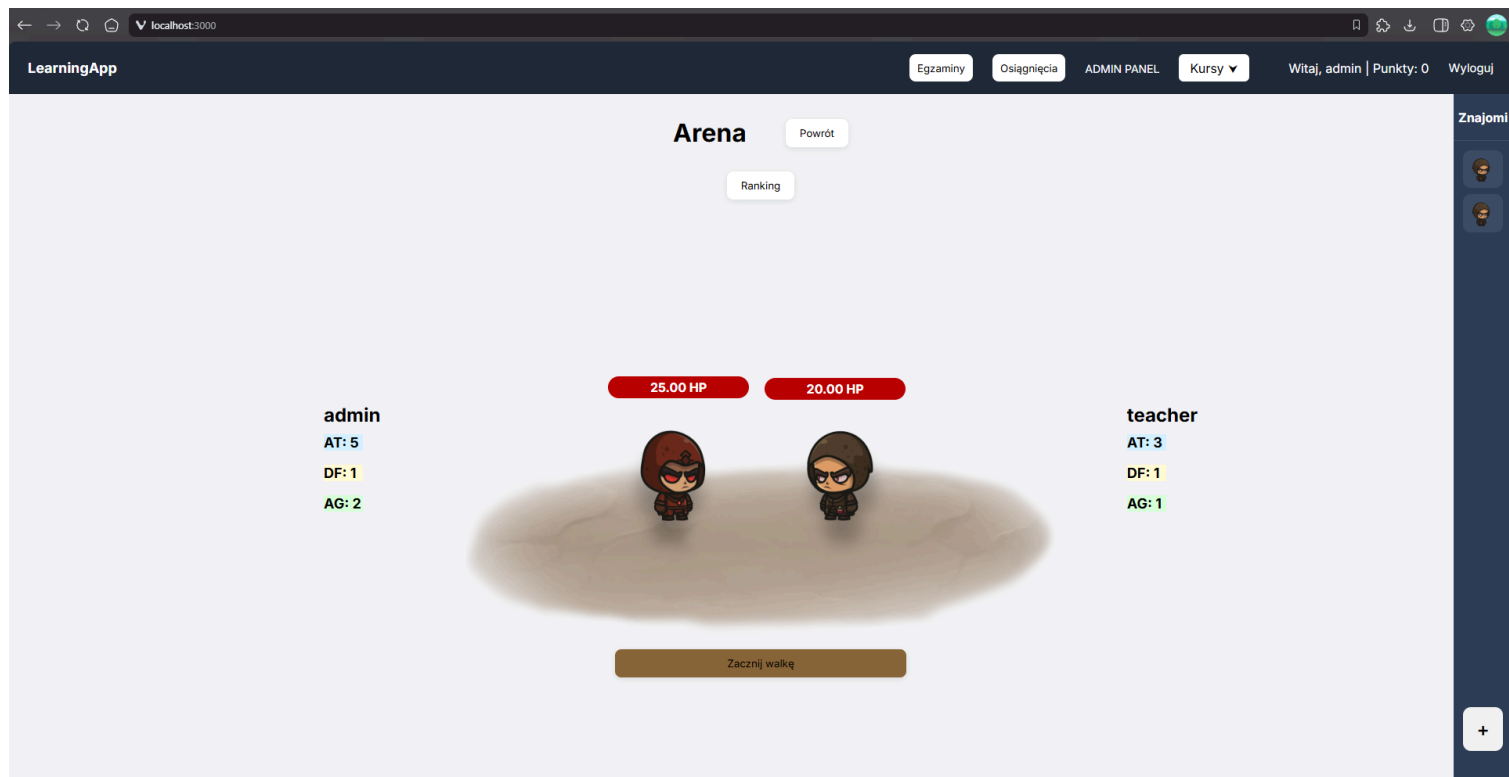
Strona rozpoczynania egzaminu wraz ze sklepem bonusów po lewej stronie



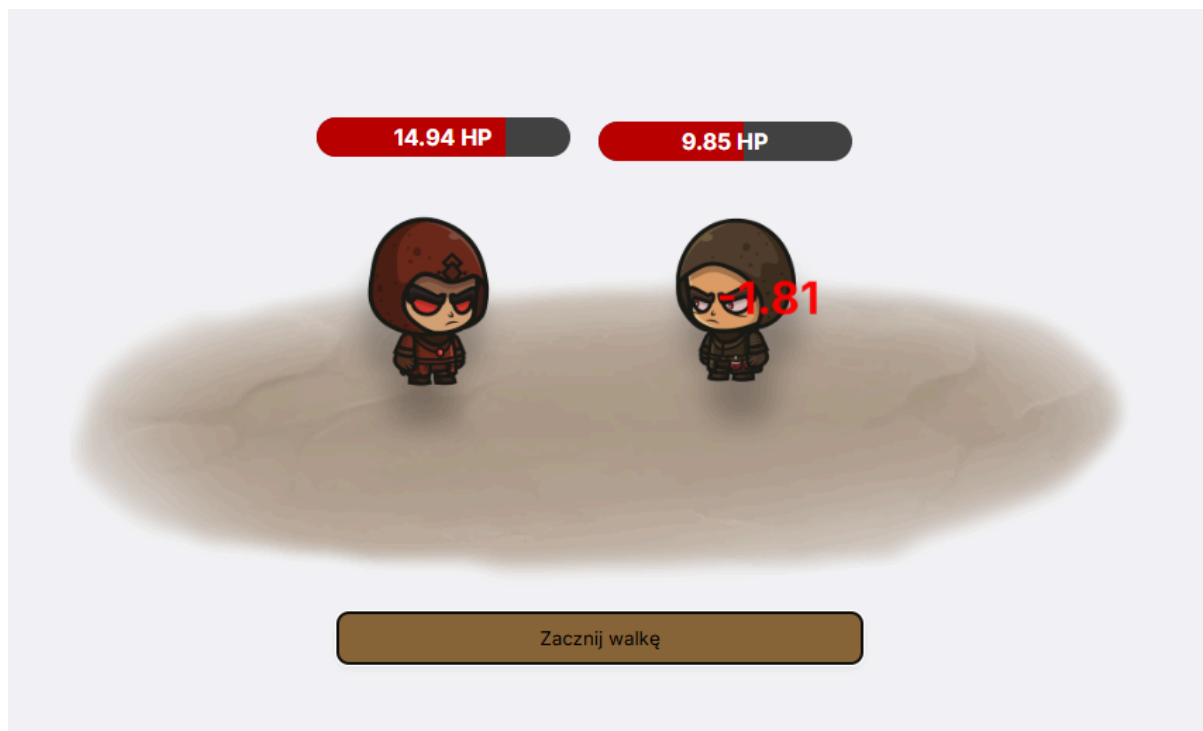
Strona egzaminu



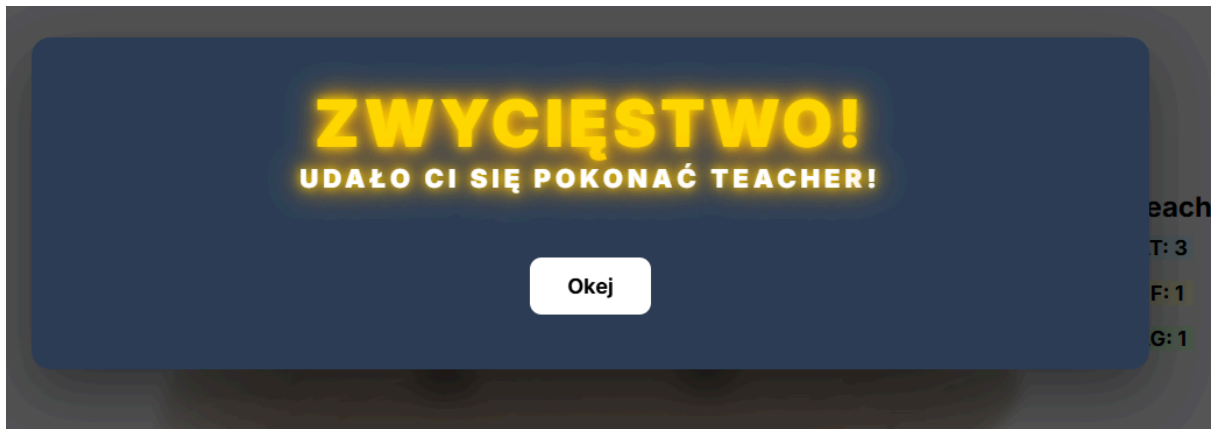
Okno z wynikiem po zakończeniu egzaminu



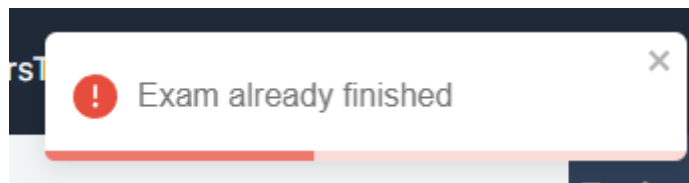
Strona z areną walk



Trwająca walka z widocznymi obrażeniami



Okno zwycięstwa w walce



Przykładowe powiadomienie wyskakujące w aplikacji

7. Podsumowanie

Stworzona aplikacja spełnia wszystkie założenia ustalone na początku semestru. Dodane funkcjonalności działają poprawnie. Pola formularzy są zabezpieczone przed podaniem nieprawidłowych wartości. Również po stronie serwera zostały dodane zabezpieczenia przed nieprawidłowo wprowadzonymi danymi. Backend działa w sposób poprawny, łączy się z bazą bez błędów. Testy manualne wykonane na aplikacji przebiegły pomyślnie. Kolejnymi możliwymi krokami w rozwoju aplikacji jest ulepszenie UI przez lepsze formatowanie stylów, dodanie możliwości edycji egzaminów dla Admina, usuwanie użytkowników ze znajomych.

8. Literatura

[1] <https://pawelkaczyk.com/grywalizacja>